CLAIMGUARD

Software Requirements Specification

Insurance Claims Processing with Fraud-Pattern Detection

Document	CLAIMGUARD_SRS
Version	2.0 — Consolidated team baseline
Date	23 October 2026
Team member 1	Hrushikesh Karnam — PES1UG24CS668
Team member 2	Nishant Hegde — PES1UG24CS681
Project context	Database Management Systems — Level 3 Experiential Learning Project
Status	Consolidated draft for team review and implementation traceability

Prepared for the ClaimGuard project team and course evaluator

Document control

Item	Description
Purpose	Defines the approved functional scope, business rules, quality requirements and acceptance criteria.
Source baseline	Consolidates ClaimGuard SRS v1.0 and teammate SRS v1.1; requirements below are the proposed common baseline.
Related document	CLAIMGUARD_ARCH v2.0 — System Architecture.
Change control	Any accepted change affecting persistent data, workflow or system boundaries must be reflected in both documents and tests.
Technology baseline	Next.js + TypeScript; Python FastAPI + Sentence Transformers; PostgreSQL 16 + pgvector; node-postgres and SQL migrations.

1. Purpose and intended audience

This Software Requirements Specification (SRS) defines what ClaimGuard must do, the constraints under which it operates, and how the team will verify the result. It is the requirements baseline for the architecture, ER diagram, relational schema, implementation, test suite and demonstration. Intended readers are the two project members, evaluator and testers.

1.1 Problem statement

Insurance claims need consistent lifecycle tracking and accountable review. Fraudulent claims can describe similar events with different wording, so exact keyword matching may miss related cases. ClaimGuard combines relational data management with semantic retrieval of previously confirmed synthetic fraud patterns to help an authorized adjuster investigate a claim.

1.2 Proposed solution

Policyholders submit and track claims; adjusters inspect a shared queue and make decisions; administrators manage users, policies, confirmed reference cases and versioned risk rules. A Python embedding service converts descriptions into normalized vectors. PostgreSQL with pgvector retrieves similar eligible reference cases, and a versioned database-side rule produces an advisory risk indicator. HIGH-risk claims are flagged for manual review. Similarity and risk are not proof of fraud, and the system never makes a final claim decision automatically.

2. Objectives

- Maintain consistent user, role, policy, claim, screening, match, review, decision, notification and audit records.
- Support the claim lifecycle from submission and screening through review, information request/response and final decision.
- Retrieve semantically similar confirmed reference cases even when the descriptions differ lexically.
- Calculate an explainable advisory risk indicator using versioned rules and flag HIGH-risk claims for mandatory human review.
- Enforce server-side role and ownership checks and database-level integrity for critical state transitions.
- Demonstrate relational and advanced DBMS features: keys, constraints, joins, aggregates, subqueries, transactions, procedures/functions, triggers, views, B-tree/GIN/HNSW indexes, full-text search and vector similarity.
- Verify the project with synthetic fixtures, repeatable tests, a documented semantic evaluation and honest limitations.

3. Scope

3.1 In scope

- Policyholder registration/sign-in; role-based access for Policyholder, Claims Adjuster and Admin.
- Admin user/role management, policy issuance, reference-case activation/deactivation and versioned risk-rule management.
- Claim submission against an owned active policy, claim tracking, status history and policyholder notifications.
- Embedding generation, top-k vector retrieval, screening attempt persistence, matched references, risk assessment and explicit failure states.
- Adjuster queue, claim detail, review start, approve/reject/request-information actions, and bounded screening retry/recovery.
- Append-only responses to information requests, notification dismissal, full-text search, operational summaries and analytical queries.
- PostgreSQL constraints, foreign keys, procedures/functions, triggers, transactions, views and indexes.

3.2 Out of scope

- Integration with real insurers, payment gateways, external identity providers or production claims systems.
- Automatic payout, settlement, denial, accusation, blacklisting or final fraud determination.
- Claims fraud model training or any guarantee of real-world fraud-detection accuracy.
- Real claimant data; all seed data, fixtures, screenshots and demonstrations must be synthetic.
- Cloud deployment, compliance certification, distributed microservices or a message broker.
- Individual adjuster assignment; adjusters use a shared review queue.
- Automatic re-embedding/re-screening of the original claim when a new information response is added, unless separately approved and tested.

4. Users and access control

Role	Permitted operations	Restrictions
Policyholder	Register/sign in; view own policies/claims/status; submit claims; answer outstanding information requests; read/dismiss own notifications.	Cannot access other users' records, internal similarity evidence, risk-rule configuration or audit logs.
Claims Adjuster	View shared queue and claim evidence; search/filter claims; start review; approve/reject/request information; retry failed/stale screening within limits.	Cannot manage users, issue policies or manage reference cases/risk rules.
Admin	Manage users/roles; issue policies; add/deactivate confirmed reference cases; create new risk-rule versions; inspect audit history and reports.	Admin operations are server-authorized and audited; Admin does not record claim decisions.

5. Assumptions and dependencies

- PostgreSQL 16 is the system of record and pgvector is installed and enabled.
- The embedding model is sentence-transformers/all-MiniLM-L6-v2; it produces L2-normalized 384-dimensional vectors. Model identity/version is recorded and cross-model comparison is prohibited.
- The model is downloaded and cached locally; first-time setup may require internet access.
- An initial administrator is provisioned through a controlled seed/setup procedure.
- New claims require an active policy, an incident date within coverage, a non-future incident date, a positive claimed amount and a nonblank description.
- Performance and semantic matching are evaluated on synthetic data; results do not establish production accuracy.

6. Functional requirements

Priority: Must = required for the agreed baseline; Should = implement after Must requirements and their tests. The identifiers below must be reused in architecture, schema traceability and test cases.

ID	Name	Requirement	Priority
FR-01	Authentication	Users can sign in with valid credentials. Passwords are stored as salted password hashes, never plaintext.	Must
FR-02	Role and ownership authorization	Every protected server operation checks the session, role and record ownership; UI hiding alone is insufficient.	Must
FR-03	Claim submission	A policyholder submits a claim against an owned active policy with valid incident date, positive amount and nonblank description.	Must
FR-04	Claim tracking	Policyholders can list and inspect only their own claims and current workflow/screening state.	Must
FR-05	Screening orchestration	Commit a new claim as SUBMITTED/PENDING before embedding inference; then screen against eligible reference cases.	Must

ID	Name	Requirement	Priority
FR-06	Similarity evidence	Persist ranked top-k matches, cosine similarity scores and model identity, or an explicit NO_MATCH result.	Must
FR-07	Versioned risk scoring	A PostgreSQL stored procedure computes an advisory risk indicator from persisted screening evidence and a versioned risk_rule.	Must
FR-08	Review workflow	An adjuster can start review and approve, reject or request information through authorized state transitions.	Must
FR-09	Approval guard	Approval is refused when screening is incomplete or required HIGH-risk review has not been started and resolved according to the workflow.	Must
FR-10	Transactional audit	Database triggers write required audit events in the same transaction as the corresponding data change.	Must
FR-11	Reference-case management	Admins add/deactivate synthetic confirmed reference cases; inactive or unconfirmed cases are excluded from retrieval.	Must
FR-12	Operational reports	Provide claim counts by status/screening state/flag and review-queue summaries.	Must
FR-13	History retention	Preserve review notes and decision history; core claim, decision and audit history cannot be casually deleted or rewritten.	Must
FR-14	Failure handling	Persist screening failure explicitly with a safe error category; failure never means LOW risk or permits approval.	Must
FR-15	Validation and state integrity	API validation and database constraints reject invalid values, invalid policy/date references and prohibited state transitions.	Must
FR-16	Automatic HIGH-risk flag	A database trigger sets review_required for HIGH risk; this flag requires human review and never causes automatic denial.	Must
FR-17	Policy issuance	Admins issue policies to policyholders with valid coverage dates/status and an audit trail.	Must
FR-18	Information response	Policyholders append responses to outstanding information requests; original submitted claim text remains unchanged.	Must
FR-19	Notifications	Users read/dismiss only their own notifications; dismissal must not delete underlying claim, decision or audit history.	Must
FR-20	Full-text search	Adjusters search claim descriptions by keyword with relevance ranking and supported filters.	Should
FR-21	Analytical reports	Provide risk distribution over time, frequent reference-pattern matches, adjuster workload/outcomes, above-average claim amounts and aged HIGH-risk claims where seed data supports them.	Should
FR-22	Retry and recovery	Adjusters retry FAILED screening; stale PENDING work is retried within configured limits. Attempts are idempotently recorded; decided claims cannot be re-screened.	Must

7. Business rules

ID	Business rule
BR-01	Policyholders access only their own policies, claims, information responses and notifications.
BR-02	New claims require an active policy; incident date must be within coverage and cannot be in the future.
BR-03	Claimed amount must be greater than zero; required text fields must be nonblank after trimming.
BR-04	Only active, confirmed reference cases with compatible embedding model identity/dimension are eligible for retrieval.
BR-05	Vector matches are ranked by cosine similarity; default top-k is 5 and is configurable/documented.
BR-06	Risk rules are versioned. Each assessment records its rule version; existing assessments are not rewritten when rules change.
BR-07	Claim submission and model inference are separate operations. PENDING/FAILED states remain visible and recoverable.
BR-08	Approval of a HIGH-risk flagged claim requires an adjuster-started review and a completed assessment. Rejection or an information request may occur before a final decision, subject to state rules.
BR-09	Audit actor identity is supplied by trusted server transaction context, not trusted from browser input.
BR-10	Original submitted claim content is immutable; extra information is appended as a response record.
BR-11	A decided claim is final: no further screening attempts, reviews, status changes or flag changes.
BR-12	Similarity scores and risk indicators are advisory; neither establishes fraud or makes a final decision.
BR-13	PENDING, FAILED or incomplete screening must never be displayed as LOW risk; approval is blocked.
BR-14	Core claim, policy, decision and audit history is retained; notification dismissal does not erase business history.

8. Non-functional requirements

ID	Quality attribute	Requirement
NFR-01	Security	Password hashing; server-side role/ownership checks; validated inputs; parameterized SQL; secrets in environment variables.
NFR-02	Integrity	Keys, constraints, triggers and transactions maintain lifecycle consistency; decision/status/audit writes are atomic.
NFR-03	Privacy	Synthetic data only in seed files, fixtures, logs, screenshots and demos.
NFR-04	Usability	Role-specific screens show clear statuses, validation failures, screening failures and advisory evidence.
NFR-05	Reliability	Screening failures are explicit/recoverable; retry is bounded/idempotent; failure never means low risk.
NFR-06	Performance	Measure common claim-list/queue queries on the synthetic dataset; record data size and timing rather than asserting an unverified SLA.
NFR-07	Maintainability	Modular monolith, numbered SQL migrations, stable requirement IDs and repeatable setup/tests.
NFR-08	Reproducibility	A developer can run Next.js, Python embedding

ID	Quality attribute	Requirement
		service and PostgreSQL/pgvector locally without requiring containers.
NFR-09	Auditability	Audit events include actor, action, target, timestamp and relevant context; ordinary application operations cannot rewrite audit history.
NFR-10	Explainability	Persist rule version and matched references so an adjuster can inspect the evidence behind each advisory indicator.

9. Technical constraints

- Web/UI/API: Next.js, React, TypeScript, Tailwind CSS and Next.js Route Handlers.
- Database: PostgreSQL 16 with pgvector; cosine distance and HNSW approximate nearest-neighbour retrieval.
- Data access: node-postgres (pg), parameterized SQL and version-controlled SQL migrations as the single schema source of truth.
- Embedding service: Python FastAPI + Sentence Transformers, all-MiniLM-L6-v2, normalized 384-dimensional vectors.
- Default top-k = 5. Risk formula and thresholds must be defined in versioned rule data and covered by tests; this SRS does not invent numeric thresholds.
- No Docker, message broker, external insurer integration or cloud deployment is required for the baseline.

10. Acceptance criteria

ID	Scenario	Expected result
AC-01	Submit a valid claim for an owned active policy.	Claim is persisted as SUBMITTED/PENDING and claim-submitted audit event is present.
AC-02	Submit invalid amount/date/policy data.	Request is rejected and database integrity remains intact.
AC-03	Request another policyholder's claim or notification.	Server denies access without revealing protected details.
AC-04	Screen a claim against eligible references.	Ranked matches persist or NO_MATCH is explicitly recorded.
AC-05	Simulate embedding-service failure.	Attempt is FAILED; claim is not LOW risk and cannot be approved.
AC-06	Attempt approval with incomplete screening or missing required HIGH-risk review.	Approval is refused and no partial decision/audit state remains.
AC-07	Record a valid adjuster decision.	Review, decision, status, audit event and notification are consistent and committed atomically.
AC-08	Run operational and analytical SQL queries on seed data.	Results match documented expected values and demonstrate required SQL features.
AC-09	Set up on a clean local machine using the README.	Developer starts all three local processes without containers.
AC-10	Screen a paraphrased description of a confirmed reference case.	Stored procedure computes risk; HIGH risk sets review_required through a trigger.
AC-11	Create a new risk-rule version and screen a new claim.	New assessment records the new version; older assessments retain theirs.
AC-12	Request information and respond as the policyholder.	Claim moves AWAITING_INFO to UNDER_REVIEW; response is append-only.
AC-13	Dismiss own notification and attempt another user's notification.	Own notification is dismissed/deleted per design; other user's record is protected.
AC-14	Search for keywords present in claim descriptions.	Matching claims are relevance-ranked and returned within the adjuster's authorization scope.
AC-15	Run semantic evaluation on paraphrased synthetic fraud and legitimate claims.	Precision/recall of the HIGH flag are recorded with dataset limitations.

11. Data model and evaluation principles

- The ER diagram captures conceptual entities and relationships; the relational schema defines columns, keys, constraints and indexes; the architecture defines components and runtime behaviour.
- Persistent entities should exist only when required by the workflow or audit/history. Query requirements may be met by indexes, views or queries rather than new tables.
- Every FR must trace to the schema, API/service operation and one or more tests. A requirement is not implemented merely because it is documented.
- Report semantic evaluation against a labelled synthetic set. No-match is not proof of legitimacy and high similarity is not proof of fraud.
- Record actual test counts, dataset size, and performance measurements only after the checked-in code is run.

12. Team responsibilities and change control

Member	SRN	Primary responsibilities (adjust to actual contribution history)
Hrushikesh Karnam	PES1UG24CS668	Database/schema and integrity review; claim workflow and screening integration; tests and documentation coordination.
Nishant Hegde	PES1UG24CS681	Architecture and API/service implementation; database procedure/trigger integration; review workflow and test coordination.

This consolidated SRS is proposed as the shared baseline. Confirm it against the approved course synopsis before submission. Any accepted change affecting data, workflow, permissions or component boundaries must update this SRS, CLAIMGUARD_ARCH, the ER diagram, relational schema, implementation and tests together.

Appendix A. System context overview

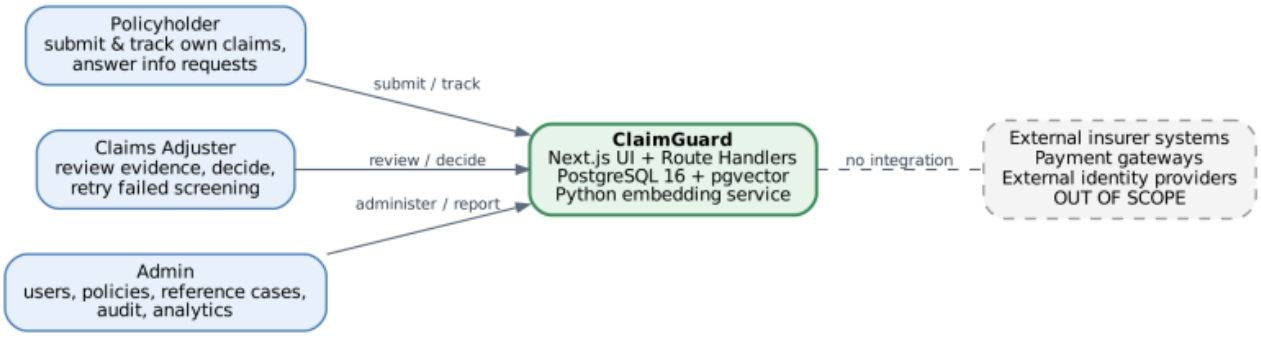


Figure 1. System context. External insurer, payment and identity-provider integrations are out of scope.